

GUIA RESUMEN — Spec Driven Development

Resumen integral para dominar SDD en ~25 minutos: teoria del libro, herramienta OpenSpec, metodologia aplicada a tu stack y auditoria de codigo generado por IA.

0 Flujo de aprendizaje general

6 PASOS · ~25 MIN

1 Teoria libro	2 Herramienta OpenSpec	3 Metodologia stack	4 Practica ejemplos	5 Auditoria Via B
----------------------	------------------------------	---------------------------	---------------------------	-------------------------

Regla de oro: "La especificacion es el contrato. El codigo cambia, la spec describe lo que el sistema HACE ahora mismo."

Paso	Que aprendes	Archivo / recurso
1. Teoria	Fases, puertas, EARS, deltas (libro SDDEquiposAgiles)	01-teoria-sdd.md + pdf/
2. Herramienta	OpenSpec CLI: init, slash commands, ciclo del cambio	03-openspec-guia-practica.md
3. Metodologia	SDD aplicado a Flutter + Clean Arch + Supabase	02-sdd-flutter-supabase.md
4. Practica	6 cambios completos de Simple a Compleja	ejemplos-cambios/
5. Auditoria	Verificar codigo escrito por IA contra la spec	06-auditoria-codigo-ia.md

1 Que es Spec Driven Development (3 min)

LIBRO · SDDEQUIPOSAGILES_V.1.PDF

SDD = escribir primero la especificacion ejecutable del comportamiento, aprobarla como contrato, y derivar de ella el codigo, los tests y la documentacion. La IA amplifica el problema: genera mucho codigo con poca comprension; la spec devuelve el control.

Los tres pilares

Pilar	Que es	Artefacto
Teoria	El libro: fases, puertas de calidad, lenguaje comun	pdf/SDDEquiposAgiles_v.1.pdf
Herramienta	OpenSpec CLI: carpeta de cambio por feature	openspec/changes/<cambio>/
Metodologia	Adaptacion al stack: que artefacto toca cada capa	02-sdd-flutter-supabase.md

El cambio de rol del desarrollador

Antes (vibe coding)	Con SDD
Prompt → codigo → rezar	Spec aprobada → codigo → verificar contra spec
"Funciona en mi maquina"	Cumple los escenarios EARS o no se mergea
Documentacion obsoleta al dia siguiente	Specs vivas: describen el sistema ACTUAL

Tres beneficios: Alineamiento antes de codear · Trazabilidad requisito→codigo→test · Onboarding y auditoria triviales.

Fase	Entregable	Puerta de calidad
1. Especificar	proposal.md + spec delta (requisitos EARS)	Puerta 1: ¿la spec es precisa y completa? ¿un dev senior la entiende sin preguntar?
2. Diseñar	design.md (ficheros, contratos Dart, backend, decisiones)	Puerta 2: ¿cada flecha del flujo tiene artefacto? ¿decisiones justificadas?
3. Tareas	tasks.md (oleadas, trazabilidad Req↔tarea↔test)	(incluida en Puerta 2)
4. Implementar	Codigo + tests + commits atomicos por tarea	Puerta 3: tests verdes, matriz llena, Clarity Gate, archive

Clarity Gate: ¿un agente distinto, con SOLO la spec, produciria codigo equivalente? Si dudas, hay supuestos ocultos → corregir la spec AHORA (todavia es barato).

Proporcionalidad: no todo merece el ritual completo

Tamano	Esfuerzo spec	Ejemplo
Simple	proposal + spec, puerta combinada	cambiar un label validado, ajuste de query
Intermedia	las 4 fases completas	nueva pantalla CRUD, add-buyers
Compleja	4 fases + Impact Report + boundaries explicitos	facturacion, delivery realtime, pagos

Cada requisito = **ID + oracion EARS + escenarios**. Sin "deberia", sin ambigüedades: el sistema DEBERA, punto.

Patron	Palabra clave	Cuando usarlo	Ejemplo (add-cart)
Event-driven	WHEN ... THE SYSTEM SHALL	evento dispara comportamiento	WHEN agrega producto con stock > 0 THEN agregar cantidad 1 y congelar precio
State-driven	WHILE <estado>	comportamiento segun estado persistente	MIENTRAS carrito tenga items, mostrar subtotal/impuesto/total
Unwanted	IF ... THEN SHALL	casos de error y excepciones	IF cupon expiro THEN "El cupon {codigo} ha expirado"
Optional	WHERE <feature>	solo si el feature esta activo	WHERE cupones activados, aplicar descuento
Complex	GIVEN ... WHEN ... THEN	precondiciones multiples	GIVEN cliente autenticado WHEN consulta su carrito THEN solo filas auth.uid()=user_id

Clasificacion de reglas → patron EARS

Tipo FADER	Significado	Patron tipico
RN (regla negocio)	invariantes del dominio (tope 50% descuento)	Unwanted / Event
RT (regla transicion)	estados y transiciones validas	State-driven
RS (regla sistema)	limites tecnicos (rate limit, tamano)	Unwanted

```
openspec/changes/add-cart/
├─ proposal.md      WHY + WHAT Changes + Impact
├─ specs/
│  └─ shopping-cart/
│     └─ spec.md    Delta: ADDED / MODIFIED / REMOVED Requirements
├─ design.md        Ficheros afectados · Contratos Dart · Backend · Decisiones
└─ tasks.md         Oleadas + trazabilidad Req-tarea-test
```

Operador delta	Semantica	Efecto en archive
## ADDED Requirements	capacidad nueva	se anaden al main spec
## MODIFIED Requirements	requisito existente cambia	reemplaza la version anterior
## REMOVED Requirements	requisito ya no aplica	se elimina del main spec

Tras el archive: **openspec/specs/** contiene las especificaciones vivas = descripcion actualizada del sistema. Nunca documentacion muerta.

Slash command	Que hace	Cuando
/opsx-explore	explorar una idea antes de especificarla	fase de discovery
/opsx-propose <nombre>	genera proposal + spec delta + design + tasks	inicio de todo cambio
/opsx-new-change	carpeta de cambio manual desde plantilla	control fino
/opsx-continue-change	retoma el cambio donde quedo	sesiones largas
/opsx-update-change	edita artifacts del cambio en curso	tras feedback de Puerta 1
/opsx-apply-change	implementa tareas oleada por oleada	Via B (IA escribe)
/opsx-verify-change	valida implementacion contra la spec	Puerta 3
/opsx-archive-change	consolida deltas en specs vivas	al aprobar todo
/opsx-sync-specs · /opsx-onboard · /opsx-bulk-archive-change	mantenimiento de specs	ocasional

CLI directa	Uso
openspec init [dir]	inicializar en un proyecto (una vez)
openspec list · openspec list --specs	ver cambios activos / capacidades
openspec validate <cambio> --strict	validar formato del cambio
openspec view · openspec change <nombre>	dashboard / detalle

Setup en 2 comandos: `npm i -g @fission-ai/openspec && openspec init` — luego reinicia el editor para cargar los slash commands (/opsx-*).

Dos estilos de arranque: /opsx-propose = modo copiloto (la IA redacta, tu apruebas). /opsx-new-change = modo artesano (esqueleto vacio; lo llenas TU y la IA refina con /opsx-update-change + validate --strict). Ambos convergen en la Puerta 1.

La spec dice	Vive en	Capa Clean Arch
entidades y calculos puros	domain/entities/*.dart	Domain
reglas de negocio (RN)	usecases / entity	Domain
contratos de datos	domain/repositories/*.dart (Either<Failure,T>)	Domain
fuentes de datos, RPCs	data/datasources/ + data/models/ (snake_case)	Data
estados UI y transiciones	presentation/cubit/ (sealed states)	Presentation
mensajes exactos de escenarios	CartError(mensaje literal), nunca en UI	toda la cadena
tablas, RLS, migraciones	supabase/migrations/NNNN_*.sql idempotente	Backend

Reglas Supabase dentro de la spec

- ✓RLS habilitada en toda tabla nueva; policies por actor × operacion declaradas en design.md \$Backend
- ✓concurrency critica (stock, secuencias) → RPC security definir atomico, no SELECT+UPDATE en cliente
- ✓aislamiento entre usuarios = escenario EARS obligatorio (GIVEN ... auth.uid() = user_id)

Oleadas estandar de tasks.md: `O1 modelos+entity → O2 datasource+repo impl → O3 usecases → O4 cubit+UI → O5 DI+rutas+migracion. Commit atomico por tarea.`

A

Scaffold IA + logica tuya

skill clean-arch-feature

B

IA escribe todo

/opsx-apply-change

Criterio	Via A (tu implementas)	Via B (IA escribe todo)
Logica critica: dinero, permisos, estados	entiendes cada linea	▲ solo con escenarios EARS exhaustivos
CRUD, formularios, brownfield quirurgico	posible overkill	terreno ideal
Aprendiendo stack o dominio	implementar ES aprender	× no aprendes nada
Specs ambiguas (falla Clarity Gate)	implementar resuelve dudas	× el error se multiplica

Regla del libro: puedes delegar la escritura, NUNCA la aprobacion de la spec (Puerta 1) ni la verificacion final (Puerta 3). La skill clean-arch-feature acepta modo openspec_change: pasa la carpeta y genera el scaffold con TODos citando REQ.

- ✓**Contratos intactos:** repository interface = design.md verbatim; Either<Failure,T> en todo; cero try/catch tragador; nada fuera de la tabla de ficheros
- ✓**Sealed states:** switch exhaustivo sin caso _; mensajes = literales EXACTOS de los escenarios ("Producto {nombre} sin stock disponible" ≠ "Stock insuficiente")
- ✓**Capa correcta:** RN en domain, no en cubit/UI; una sola fuente de verdad cliente/servidor; decisiones D1..Dn respetadas
- ✓**Supabase ▲:** RLS en todas las tablas nuevas; policies = design.md; RPC security definer revisado linea por linea; cero service_role en cliente; migracion idempotente e igual al design
- ✓**Tests contra spec:** 1 test por Scenario citando REQ; asserts usan literales; cero tests siempre-verdes
- ✓**Trazabilidad:** matriz llena; cero UnimplementedError() huérfanos; commit atomico por tarea
- ✓**Red flags:** diff fuera de alcance; dependencias nuevas sin justificar; over-engineering; boundaries violados ("No desactivar RLS jamas")

Ciclo por oleada: agente aplica → tu auditas diff vs spec → cumple ✓ sigue / no cumple → fix citando REQ+escenario. Si el bug era de la SPEC, corrígela primero (/opsx-update-change) y re-aplica.

Antes (FADER / Scrum)	Ahora (SDD / OpenSpec)
Descomposicion de feature	Carpeta de cambio (proposal + specs + design + tasks)
Historias de usuario + criterios Gherkin	Requirements con ID + escenarios EARS
Definition of Done / sprint review	Puertas de calidad (Puerta 1, 2, 3)
Product Backlog refinado	Main specs (openspec/specs/) — sistema actual
Nuevo sprint para cambiar algo	Deltas ADDED / MODIFIED / REMOVED
Mapeo FADER a capas	design.md \$Ficheros afectados (Elemento Capa Archivo Req)

Cambio	Capacidad	Complejidad	Lo que demuestra
add-cart	shopping-cart	Intermedia ★ walkthrough	flujo completo paso a paso; RPC atomico stock; precio congelado; RLS por usuario
add-buyers	buyers-management	Simple → Intermedia ★★	puerta combinada; aprobar/liberar tickets; minimo ritual suficiente
approve-reservas	appointments	Compleja ★★★	doble confirmacion; reagendar con politica 24h; pg_cron; realtime
add-elearning-progress	enrollment-progress	Intermedia ★★	gating de lecciones via RLS; vista de progreso; trigger auto-completada
add-facturacion	invoicing	Compleja ★★★	maquina de estados invoice; secuencias fiscales; RPC transaccional; notas credito
add-delivery-seguimiento	delivery-tracking	Compleja ★★★	GPS realtime broadcast; tarifa PostGIS; timeout de entrega con cron

Como practicar: lee add-cart completo junto a 02-sdd-flutter-supabase.md → copia la estructura a openspec/changes/ en tu proyecto → adapta. Luego intenta escribir tu propio cambio ANTES de pedirle la spec a la IA.

